

まずはこちらを読み、

継承における「基本クラスの内容を派生クラス内で利用する方法」を学びましょう。

その 1.

基本クラスのコンストラクタの機能を派生クラスで活用する方法

⇒ base()、base(引数) などを継承する形で利用する

Kart.cs のコンストラクタの例

```
//基本クラスのコンストラクタの例
public Kart(int weight, int speed)
{
    this.weight = weight;
    this.speed = speed;
}
```

SkyKart.cs のコンストラクタの例

```
private int flyingPower; //派生クラス独特の変数

//派生クラスでコンストラクタを用意したい場合
public SkyKart(int weight, int speed, int flyingPower) : base(weight, speed)
{
    this.flyingPower = flyingPower; //基本クラスにないものは追加で設定する
}
```

基本クラスの
コンストラクタを利用
base(weight, speed)

その 2.

基本クラスにあった変数やプロパティ、メソッドを派生クラスで活用する方法

⇒ base. 変数名、base. プロパティ名、base. メソッド名などで利用する

※ただし private 以外のアクセス修飾子であることが条件

```
public override void Force() 例
{
    base.Force(); //基本クラスと同じことをしてみる
    //追加したいことなどがあれば足す
    Console.WriteLine($"重量は{base.Weight}");
    Console.WriteLine($"スピードは{base.Speed}");
    //基本クラスの変数が private で「base. 変数名という使い方が NG」
    //でも、base. プロパティなら OK の可能性
}
```

次ページから課題です。課題は 4 プロジェクト分です。

GitHub に模範解答が掲載しておりますが、自分で試行錯誤する時間は大事です。

最初は教科書を振り返りながら自分でやってみて、

どうしても行き詰るようなら模範解答を参考にしてください。

itaku202609_materials / 授業資料 / プログラミング演習 / ドットNET / 0917Practice /

Dyna-Rise CS演習の模範解答0917

Name	Last commit message
..	
.vs	CS演習の模範解答0917
EmployeeSystem	CS演習の模範解答0917
GameCharacter	CS演習の模範解答0917
ProductInventory	CS演習の模範解答0917
ShapeCalculator	CS演習の模範解答0917
0917Practice.slnx	CS演習の模範解答0917

それぞれ対応する
プロジェクトフォルダの中の
.cs ファイルが模範解答です

次の課題の設計や Main のコンソール結果を目指して
課題ごとにプロジェクトを作成してみましょう。

もし課題文に誤植や問題が見つかったら
ご自分なりに解釈して処理してくださって OK です

課題 1

社員管理システム プロジェクト名: EmployeeSystem

基本クラス Employee

メンバ変数: private string name(従業員名)
private int salary (基本給)

コンストラクタ: なし

アクセサとなるメソッド: 基本給をセットできるSetSalary (最低でもsalaryは20万円となるように)
基本給を参照できるGetSalary

従業員名と給与をコンソール表示するメソッド: ShowInfo()

派生クラス Manager

独自のメンバ変数: private float bounusRate = 0.2f (ボーナスの倍率)

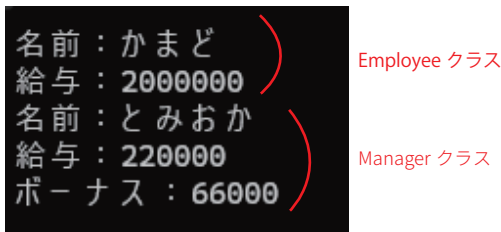
コンストラクタ: なし

アクセサとなるメソッド: ボーナスの倍率指定できるSetBousRate

オーバーライドされるメソッド: ShowInfo()

⇒もとの機能に加えて、ボーナス倍率*基本給によるボーナスも表示

Mainメソッドの結果: 同じコンソール出力を目指してコーディングしてみましょう



```
名前 : かまど  
給与 : 2000000  
名前 : とみおか  
給与 : 220000  
ボ ー ナ ス : 66000
```

Employee クラス

Manager クラス

課題 2

図形面積計算システム プロジェクト名: ShapeCalculator

基本クラス Shape

メンバ変数: private string info(図形の種類)

コンストラクタ: 引数なし(infoは"無形")、引数あり 両方用意

アクセサとなるプロパティ Info: setもgetもできる

面積を求めるメソッド: GetArea() ※Shapeクラスにおいては「○○の面積はなし」と表示

派生クラス① Square

独自のメンバ変数: private float bottom (底辺)

private float height (高さ)

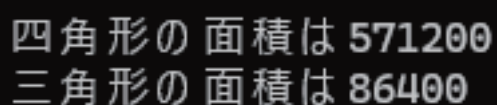
コンストラクタ: 引数あり<infoが"四角形"、底辺と高さを指定>、引数なし<infoが"四角形"、底辺と高さは100>

アクセサなし

オーバーライドされるメソッド: GetArea() 四角形の面積を求める

派生クラス② Triangle ※派生①とほぼ同じ構成で infoが"三角形"、GetAreaの面積も三角形の面積

Mainメソッドの結果: 同じコンソール出力を目指してコーディングしてみましょう



```
四角形の面積は 571200  
三角形の面積は 86400
```

Square クラス 底辺 840、高さ 680

Triangle クラス 底辺 480、高さ 360

課題 3

ゲームキャラクター管理 プロジェクト名: GameCharacter

基本クラス Character

メンバ変数: private string name(名前)

private int hp(体力)

コンストラクタ: 引数なし (nameは"名無し"、体力100)、引数あり 両方用意

アクセサとなるプロパティ: Name (8文字以下ならセット可能、getあり)

Hp (0以下なら1に修正、getあり)

攻撃メソッド: Attack() ※「名前は攻撃した!」と表示

ステータス表示メソッド: DisplayStatus() 名前と体力をコンソール表示

派生クラス Warrior

独自のメンバ変数: なし

コンストラクタ: 引数あり、引数なし 両方用意、それぞれ機能はbaseと同じ

アクセサ: なし

追加実装されるメソッド: HyperAttack()

「○○の魂の一撃!」という表示のあと

体力が10より上なら 体力を10削る「体力が10減った!残り(○○)」

そうでなければ「不発!体力が足りない!」

Mainメソッドの結果: 同じコンソール出力を目指してコーディングしてみましょう

```
名前 : ごんたろー
体力 : 100
ごんたろー は 攻撃した !
名前 : じんのすけ
体力 : 145
じんのすけの魂の一撃 !
体力が10減った ! 残り (135)
じんのすけの魂の一撃 !
体力が10減った ! 残り (125)
じんのすけの魂の一撃 !
体力が10減った ! 残り (115)
じんのすけの魂の一撃 !
体力が10減った ! 残り (105)
じんのすけの魂の一撃 !
体力が10減った ! 残り (95)
じんのすけの魂の一撃 !
体力が10減った ! 残り (85)
じんのすけの魂の一撃 !
体力が10減った ! 残り (75)
じんのすけの魂の一撃 !
体力が10減った ! 残り (65)
じんのすけの魂の一撃 !
体力が10減った ! 残り (55)
じんのすけの魂の一撃 !
体力が10減った ! 残り (45)
じんのすけの魂の一撃 !
体力が10減った ! 残り (35)
じんのすけの魂の一撃 !
体力が10減った ! 残り (25)
じんのすけの魂の一撃 !
体力が10減った ! 残り (15)
じんのすけの魂の一撃 !
体力が10減った ! 残り (5)
```

Character クラス

Warrior クラス
HyperAttack() を発動できるかぎり
ループして発動
不発の条件がきたらループを抜けて終了

課題 4

商品在庫管理 プロジェクト名: ProductInventory

基本クラス Product

メンバ変数: private string name(名前)

private int price(値段)

private int number(在庫数)

コンストラクタ: 引数なし (nameは"未登録"、値段100、在庫数0)、引数あり 両方用意

アクセサとなるプロパティ: Name (setあり、getあり)

Price(0未満なら0に修正、getあり)

Number(0未満なら0に修正、getあり)

在庫リスト表示メソッド: GetStockList() 商品名、値段、在庫数を1行で表示

派生クラス Food

独自のメンバ変数: private int untilExpiration(残り消費期限日数)

コンストラクタ: 引数あり<baseの引数ありに加えて、残り消費期限日数の設定>、

引数なし<残り消費期限日数が10に設定>

両方用意

アクセサ: なし

オーバーライドされるメソッド: GetStockList()

baseの機能に加え、「消費期限まであと〇日」が2行目に追加表示される

Mainメソッドの結果: 同じコンソール出力を目指してコーディングしてみましょう

```
商品名 : ざるそば、値段500円、在庫数5個
消費期限まであと3日
商品名 : ざるうどん、値段480円、在庫数3個
消費期限まであと3日
商品名 : 天井、値段680円、在庫数3個
消費期限まであと2日
商品名 : からあげ弁当、値段580円、在庫数2個
消費期限まであと1日
商品名 : ハンバーグ弁当、値段480円、在庫数1個
消費期限まであと1日
商品名 : 幕ノ内弁当、値段680円、在庫数3個
消費期限まであと2日
```

Food クラスのみ使用

見本と同じだけの情報を Food クラスの配列に格納
for あるいは foreach を使って見本と同じように表示

課題が早めに終わったら

- ・テキストの予習復習
- ・履歴書・職務経歴の編集
- ・求人情報の検索
- ・課題内容を自分なりにアレンジして研究

など、余った時間をうまく使いましょう！